LAPORAN PRAKTIKUM

Modul 6: Generate Data Otomatis & Authentication



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 6 ini saya kerjakan untuk menambahkan autentikasi pada project LaporBanjir yang sudah saya buat sebelumnya. Pada Modul 4, aplikasi ini hanya punya form pelaporan dan daftar laporan tanpa pembatasan akses. Semua orang bisa membuka halaman pengelolaan. Pada modul ini saya memakai Laravel Breeze supaya proses register, login, logout, dan edit profil tidak perlu ditulis dari nol. Fokus praktikumnya ada pada pemasangan Breeze, pemahaman alur autentikasi, dan penyesuaian halaman LaporBanjir agar tetap berjalan.

Tujuan yang ingin saya capai pada praktikum ini adalah:

1. Memahami cara kerja authentication, session, hashing password, dan middleware auth pada Laravel.

2. Memasang Laravel Breeze dan menjalankan perintah instalasinya pada project LaporBanjir.

3. Menelusuri route, controller, request, dan view autentikasi yang dibuat Breeze, mulai dari register, login, sampai logout.

4. Menghubungkan autentikasi Breeze dengan halaman LaporBanjir tanpa mengubah CSS default Breeze.

5. Menentukan halaman yang boleh dibuka warga tanpa akun dan halaman yang hanya boleh dibuka petugas.

6. Membuat PetugasSeeder dengan Faker untuk menyiapkan lima akun dummy dan satu akun admin.

7. Menguji register, login, logout, profile, redirect halaman terlindungi, dan penyimpanan password dalam bentuk hash.

2. Alat dan Bahan

Praktikum ini saya kerjakan di laptop pribadi dengan Windows 11 Pro 64-bit, prosesor Intel Core i7-13650HX, dan RAM 16 GB. Project berada di D:\xampp\htdocs\PABW\Praktikum_6\Praktikum_TA_6. Aplikasi dijalankan dengan php artisan serve pada port 8123 dan memakai database db_laporbanjir_modul6. XAMPP sudah terpasang di D:\xampp dan dipakai untuk Apache serta MariaDB.

2.1 Perangkat Lunak

Tabel 2.1 merangkum perangkat lunak dan versi yang saya pakai. Versi ini saya periksa langsung dari terminal dan halaman phpMyAdmin, bukan hanya mengikuti angka yang tertulis di modul.

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MariaDB)	Terpasang pada D:\xampp
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel	12.70.0
Laravel Breeze	2.4.2
Node.js	25.6.0
NPM	11.8.0
Vite	7.3.7
MariaDB	10.4.32
phpMyAdmin	5.2.1
Git	2.53.0.windows.1
Google Chrome	155.0.8059.82
Visual Studio Code	Editor kode dan terminal

3. Langkah-Langkah Praktikum

Project TA6 adalah kelanjutan LaporBanjir dari Modul 4. Struktur utamanya masih sama: model Laporan, LaporanController, form publik, halaman konfirmasi, dan daftar laporan. Karena instruksi tugas meminta Breeze, saya tidak menulis controller autentikasi manual. Saya memasang Breeze, lalu menyesuaikan route dan view LaporBanjir supaya memakai layout serta komponen Breeze. Sebelum instalasi, saya mencatat file LaporBanjir yang akan tertimpa, terutama routes/web.php dan layouts/app.blade.php. Setelah Breeze selesai, kedua file itu saya sesuaikan lagi agar halaman lama tetap bisa dipakai.

3.1 Instalasi Laravel Breeze

Langkah pertama adalah memasang package Breeze sebagai dev dependency. Perintah composer require laravel/breeze --dev menambahkan package tersebut ke composer.json. Setelah itu, php artisan breeze:install blade membuat route autentikasi, controller, request, view, dan test. Saya memilih stack Blade karena LaporBanjir memang memakai Blade, bukan React atau Vue. Perintah npm install menyiapkan dependency frontend, lalu npm run build mengompilasi Tailwind dan JavaScript ke folder public/build. Tanpa langkah build, halaman Breeze bisa error karena manifest Vite tidak ditemukan.

```
composer require laravel/breeze
php artisan breeze:install blade
npm install
npm run build
```

Setelah instalasi, muncul folder app/Http/Controllers/Auth, app/Http/Requests/Auth, resources/views/auth, dan resources/views/layouts. File resources/css/app.css tetap berisi tiga directive Tailwind default, yaitu @tailwind base, @tailwind components, dan @tailwind utilities. Saya tidak mengubah isi file itu karena ketentuan praktikum meminta CSS Breeze tetap default. Perubahan tampilan hanya lewat class Tailwind di file Blade.

3.2 Konfigurasi Database dan Migration

Konfigurasi database ada di file .env. Saya mengubah DB_DATABASE menjadi db_laporbanjir_modul6, DB_USERNAME menjadi root, dan DB_PASSWORD dikosongkan sesuai pengaturan XAMPP. APP_FAKER_LOCALE saya isi id_ID agar nama dummy terasa lokal. Setelah database dibuat di phpMyAdmin, saya menjalankan php artisan migrate:fresh --seed. Perintah ini menghapus tabel lama, menjalankan ulang migration, lalu mengisi data dari seeder. Tabel yang terbentuk antara lain users, password_reset_tokens, sessions, cache, jobs, dan laporan.

```
APP_NAME=LaporBanjir
APP_FAKER_LOCALE=id_ID

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laporbanjir_modul6
DB_USERNAME=root
DB_PASSWORD=
```

Tabel users menyimpan name, email, password, dan timestamps. Kolom password tidak menyimpan teks asli. Breeze memakai Hash::make() sehingga isinya menjadi hash bcrypt dengan awalan $2y$12$. Tabel sessions dipakai untuk menyimpan sesi login, sedangkan tabel laporan tetap memakai struktur dari Modul 4.

3.3 Route Autentikasi dan Proteksi Halaman

Breeze memisahkan route autentikasi ke routes/auth.php. File routes/web.php memanggil file itu di bagian bawah. Untuk LaporBanjir, saya mengatur tiga kelompok route. Pertama, route publik: laporan.create, laporan.store, dan laporan.konfirmasi. Warga tidak perlu akun untuk melapor. Kedua, route yang dilindungi middleware auth: dashboard, laporan.index, profile.edit, profile.update, dan profile.destroy. Ketiga, route guest dari Breeze untuk login, register, forgot password, dan reset password.

```php
<?php

use App\Http\Controllers\LaporanController;
use App\Http\Controllers\ProfileController;
use App\Models\Laporan;
use Illuminate\Support\Facades\Route;

// Halaman awal diarahkan ke form pelaporan yang dapat diakses tanpa login.
Route::get('/', function () {
    return redirect()->route('laporan.create');
});

// ==================== Route publik (warga) ====================
Route::get('/laporan/create', [LaporanController::class, 'create'])->name('laporan.create');
Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');
Route::get('/laporan/{laporan}/konfirmasi', [LaporanController::class, 'konfirmasi'])->name('laporan.konfirmasi');

// ==================== Route yang dilindungi middleware auth ====================
Route::get('/dashboard', function () {
    return view('dashboard', [
        'totalLaporan' => Laporan::count(),
        'laporanTerbaru' => Laporan::latest()->take(5)->get(),
    ]);
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    // Halaman daftar laporan hanya boleh dibuka oleh petugas yang sudah login.
    Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');

    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

Route dashboard juga memakai middleware verified. Di project ini akun dibuat lewat register atau seeder dan fitur verifikasi email tidak diaktifkan, jadi middleware itu tidak mengganggu alur. Route login diberi nama login karena middleware auth memakai nama tersebut untuk mengarahkan pengunjung ke halaman login. Route logout memakai method POST dan middleware auth. Saya tidak mengubah nama route itu supaya komponen Breeze tetap bekerja.



```php
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);

    Route::get('forgot-password', [PasswordResetLinkController::class, 'create'])
        ->name('password.request');

    Route::post('forgot-password', [PasswordResetLinkController::class, 'store'])
        ->name('password.email');

    Route::get('reset-password/{token}', [NewPasswordController::class, 'create'])
        ->name('password.reset');

    Route::post('reset-password', [NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification',
[EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');

    Route::get('confirm-password', [ConfirmablePasswordController::class, 'show'])
        ->name('password.confirm');

    Route::post('confirm-password', [ConfirmablePasswordController::class, 'store']);

    Route::put('password', [PasswordController::class, 'update'])-
>name('password.update');

    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

Untuk membuktikan proteksi, saya membuka /laporan tanpa login. Browser berpindah ke /login. Setelah login sebagai admin, halaman daftar laporan bisa dibuka. Form /laporan/create tetap bisa diakses tanpa login. Ini yang saya inginkan karena warga tidak perlu membuat akun hanya untuk mengirim laporan.

3.4 Controller dan Request Autentikasi

Breeze membuat controller register, login, logout, password reset, email verification, dan profile. Saya tidak menulis ulang controller tersebut, tetapi saya membaca alurnya. RegisteredUserController memvalidasi name, email, dan password. Email harus unik. Password memakai aturan Password::defaults() dan harus sama dengan password_confirmation. Setelah

valid, password di-hash dengan Hash::make(), user disimpan, lalu Auth::login() dipanggil dan session di-regenerate. Pengguna langsung diarahkan ke dashboard.



```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}
```

AuthenticatedSessionController menangani login dan logout. Method store memanggil LoginRequest, lalu Auth::attempt() dilakukan di request tersebut. Setelah berhasil, session di-regenerate untuk mencegah session fixation. Logout menjalankan Auth::logout(), invalidate session, dan regenerate CSRF token. Karena logout memakai POST, setiap tombol logout harus berada di form dengan @csrf.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    /**
     * Display the login view.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard', absolute: false));
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

LoginRequest berisi aturan validasi dan rate limiter. Kalau email atau password salah, muncul pesan These credentials do not match our records. Rate limiter membatasi lima percobaan per kombinasi email dan IP. Setelah batas tercapai, muncul pesan Too many login attempts. Mekanisme ini membantu mengurangi tebakan password. Kode LoginRequest saya bagi menjadi dua gambar agar teksnya tetap terbaca.

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Auth\Events\Lockout;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }

    /**
     * Attempt to authenticate the request's credentials.
     *
     * @throws ValidationException
     */
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();
```

```php

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember')))
    {
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => trans('auth.failed'),
            ]);
        }

        RateLimiter::clear($this->throttleKey());
    }

    /**
     * Ensure the login request is not rate limited.
     *
     * @throws ValidationException
     */
    public function ensureIsNotRateLimited(): void
    {
        if (! RateLimiter::tooManyAttempts($this->throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));

        $seconds = RateLimiter::availableIn($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds / 60),
            ]),
        ]);
    }

    /**
     * Get the rate limiting throttle key for the request.
     */
    public function throttleKey(): string
    {
        return Str::transliterate(Str::lower($this->string('email')).'|'.$this->ip());
    }
}
```

3.5 View Autentikasi Breeze

View autentikasi ada di resources/views/auth. Register memakai guest layout dan komponen input Breeze. Field-nya Name, Email, Password, dan Confirm Password. Login memakai Email, Password, Remember me, dan tautan Forgot your password. Semua class Tailwind berasal dari Breeze. Saya membiarkan struktur form autentikasi tetap default. Perubahan tampilan saya lakukan pada halaman LaporBanjir, dashboard, profil, dan navigasi.

```blade
<x-guest-layout>
    <form method="POST" action="{{ route('register') }}">
        @csrf

        <!-- Name -->
        <div>
            <x-input-label for="name" :value="__('Name')" />
            <x-text-input id="name" class="block mt-1 w-full" type="text" name="name" :value="old('name')" required autofocus autocomplete="name" />
            <x-input-error :messages="$errors->get('name')" class="mt-2" />
        </div>

        <!-- Email Address -->
        <div class="mt-4">
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <!-- Password -->
        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />

            <x-text-input id="password" class="block mt-1 w-full"
                            type="password"
                            name="password"
                            required autocomplete="new-password" />

            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <!-- Confirm Password -->
        <div class="mt-4">
            <x-input-label for="password_confirmation" :value="__('Confirm Password')" />

            <x-text-input id="password_confirmation" class="block mt-1 w-full"
                            type="password"
                            name="password_confirmation" required autocomplete="new-password" />

            <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
        </div>

        <div class="flex items-center justify-end mt-4">
            <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500" href="{{ route('login') }}">
                {{ __('Already registered?') }}
            </a>

            <x-primary-button class="ms-4">
                {{ __('Register') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

```blade
<x-guest-layout>
    <!-- Session Status -->
    <x-auth-session-status class="mb-4" :status="session('status')" />

    <form method="POST" action="{{ route('login') }}">
        @csrf

        <!-- Email Address -->
        <div>
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autofocus autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <!-- Password -->
        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />

            <x-text-input id="password" class="block mt-1 w-full"
                            type="password"
                            name="password"
                            required autocomplete="current-password" />

            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <!-- Remember Me -->
        <div class="block mt-4">
            <label for="remember_me" class="inline-flex items-center">
                <input id="remember_me" type="checkbox" class="rounded border-gray-300 text-indigo-600 shadow-sm focus:ring-indigo-500" name="remember">
                <span class="ms-2 text-sm text-gray-600">{{ __('Remember me') }}</span>
            </label>
        </div>

        <div class="flex items-center justify-end mt-4">
            @if (Route::has('password.request'))
                <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500" href="{{ route('password.request') }}">
                    {{ __('Forgot your password?') }}
                </a>
            @endif

            <x-primary-button class="ms-3">
                {{ __('Log in') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

Dashboard memakai x-app-layout. Di dalamnya saya menambahkan ringkasan total laporan, tombol akses cepat ke Daftar Laporan dan Buat Laporan, serta tabel lima laporan terbaru. Data diambil dari Laporan::count() dan Laporan::latest()->take(5)->get(). Tampilan dashboard tetap memakai card putih, border abu-abu, dan shadow kecil dari Breeze. Tidak ada CSS tambahan.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8 space-y-6">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    <p class="text-lg font-medium">
                        Selamat datang, {{ Auth::user()->name }}.
                    </p>
                    <p class="mt-1 text-sm text-gray-600">
                        Anda berhasil login sebagai petugas BPBD Kabupaten Bandung.
                    </p>
                </div>
            </div>

            <div class="grid gap-6 sm:grid-cols-2">
                <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                    <div class="p-6">
                        <p class="text-sm font-medium text-gray-500">Total Laporan Banjir</p>
                        <p class="mt-2 text-3xl font-semibold text-gray-900">{{ $totalLaporan }}</p>
                    </div>
                </div>

                <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                    <div class="p-6">
                        <p class="text-sm font-medium text-gray-500">Akses Cepat</p>
                        <div class="mt-3 flex flex-wrap gap-3">
                            <a href="{{ route('laporan.index') }}"
                               class="inline-flex items-center px-4 py-2 bg-gray-800 border border-transparent rounded-md font-semibold text-xs text-white uppercase tracking-widest hover:bg-gray-700 focus:bg-gray-700 active:bg-gray-900 focus:outline-none focus:ring-2 focus:ring-indigo-500 focus:ring-offset-2 transition ease-in-out duration-150">
                                Daftar Laporan
                            </a>
                            <a href="{{ route('laporan.create') }}"
                               class="inline-flex items-center px-4 py-2 bg-white border border-gray-300 rounded-md font-semibold text-xs text-gray-700 uppercase tracking-widest shadow-sm hover:bg-gray-50 focus:outline-none focus:ring-2 focus:ring-indigo-500 focus:ring-offset-2 disabled:opacity-25 transition ease-in-out duration-150">
                                Buat Laporan
                            </a>
                        </div>
```

```
                    </div>
                </div>
            </div>

            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6">
                    <h3 class="text-lg font-medium text-gray-900">Laporan Terbaru</h3>

                    @if ($laporanTerbaru->isEmpty())
                        <p class="mt-3 text-sm text-gray-600">Belum ada laporan banjir yang masuk.</p>
                    @else
                        <div class="mt-4 overflow-x-auto">
                            <table class="min-w-full divide-y divide-gray-200 text-sm">
                                <thead class="bg-gray-50">
                                    <tr>
                                        <th class="px-4 py-3 text-left font-medium text-gray-500 uppercase tracking-wider">Lokasi</th>
                                        <th class="px-4 py-3 text-left font-medium text-gray-500 uppercase tracking-wider">Pelapor</th>
                                        <th class="px-4 py-3 text-left font-medium text-gray-500 uppercase tracking-wider">Tinggi Genangan</th>
                                        <th class="px-4 py-3 text-left font-medium text-gray-500 uppercase tracking-wider">Tanggal</th>
                                    </tr>
                                </thead>
                                <tbody class="divide-y divide-gray-200 bg-white">
                                    @foreach ($laporanTerbaru as $laporan)
                                        <tr>
                                            <td class="px-4 py-3 text-gray-900">{{ $laporan->lokasi }}</td>
                                            <td class="px-4 py-3 text-gray-700">{{ $laporan->nama_pelapor }}</td>
                                            <td class="px-4 py-3 text-gray-700">{{ $laporan->tinggi_genangan }} cm</td>
                                            <td class="px-4 py-3 text-gray-700">
                                                {{ \Illuminate\Support\Carbon::parse($laporan->tanggal_kejadian)->format('d-m-Y') }}
                                            </td>
                                        </tr>
                                    @endforeach
                                </tbody>
                            </table>
                        </div>
                    @endif
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

Halaman profil juga memakai layout Breeze. Saya menambahkan card Informasi Akun Petugas yang menampilkan nama, email, dan tanggal akun dibuat dari Auth::user(). Bagian update password dan delete account tetap seperti bawaan Breeze. Nilai di card saya buat rata kiri supaya mudah dibaca, tetapi class yang dipakai masih Tailwind default.

3.6 Integrasi LaporBanjir dengan Layout Breeze

LaporBanjir dari Modul 4 memakai layout sendiri. Setelah Breeze masuk, saya memindahkan halaman LaporBanjir ke x-app-layout agar navigasi dan style-nya konsisten. LaporanController tidak banyak berubah. Method create menampilkan form, store memvalidasi dan menyimpan

laporan, index mengambil seluruh laporan dengan Laporan::all(), dan konfirmasi menampilkan detail laporan yang baru dikirim.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;

class LaporanController extends Controller
{
    /**
     * Halaman daftar laporan banjir.
     * Data diambil dari database menggunakan Eloquent (Laporan::all()).
     */
    public function index()
    {
        $laporans = Laporan::all();

        return view('laporan.index', compact('laporans'));
    }

    /**
     * Halaman form pelaporan banjir.
     */
    public function create()
    {
        return view('laporan.form');
    }

    /**
     * Memproses data form yang dikirim melalui method POST,
     * lalu menyimpannya ke tabel laporan menggunakan Laporan::create().
     */
    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama_pelapor'     => 'required|string|max:100',
            'lokasi'           => 'required|string|max:150',
            'tinggi_genangan'  => 'required|numeric|min:0',
            'tanggal_kejadian' => 'required|date',
        ]);

        // Menyimpan data ke database melalui Eloquent.
        $laporan = Laporan::create($validated);

        return redirect()
            ->route('laporan.konfirmasi', $laporan)
            ->with('success', 'Laporan berhasil disimpan ke database.');
    }

    /**
     * Halaman konfirmasi yang menampilkan kembali data laporan
     * yang baru saja tersimpan di database.
     */
    public function konfirmasi(Laporan $laporan)
    {
        return view('laporan.konfirmasi', compact('laporan'));
    }
}
```

Form publik memakai komponen input dan tombol Breeze. Field yang dipakai: nama pelapor, lokasi kejadian, tinggi genangan air dalam cm, dan tanggal kejadian. Validasi tetap di controller. Setelah data disimpan, pengguna diarahkan ke halaman konfirmasi, bukan ke daftar laporan, karena daftar laporan hanya untuk petugas.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Form Pelaporan Banjir') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    <p class="text-sm text-gray-600">
                        Warga dapat mengirim laporan tanpa harus login. Isi data di bawah ini sesuai kondisi banjir di wilayah Anda.
                    </p>

                    <form action="{{ route('laporan.store') }}" method="POST" class="mt-6 space-y-4">
                        @csrf

                        <div>
                            <x-input-label for="nama_pelapor" :value="__('Nama Pelapor')" />
                            <x-text-input id="nama_pelapor" class="block mt-1 w-full" type="text" name="nama_pelapor"
                                          :value="old('nama_pelapor')" placeholder="Contoh: Ahmad Fauzi" required autofocus />
                            <x-input-error :messages="$errors->get('nama_pelapor')" class="mt-2" />
                        </div>

                        <div>
                            <x-input-label for="lokasi" :value="__('Lokasi Kejadian')" />
                            <x-text-input id="lokasi" class="block mt-1 w-full" type="text" name="lokasi"
                                          :value="old('lokasi')" placeholder="Contoh: Kp. Cieunteung, Kec. Baleendah" required />
                            <x-input-error :messages="$errors->get('lokasi')" class="mt-2" />
                        </div>

                        <div>
                            <x-input-label for="tinggi_genangan" :value="__('Tinggi Genangan Air (cm)')" />
                            <x-text-input id="tinggi_genangan" class="block mt-1 w-full" type="number" name="tinggi_genangan"
                                          :value="old('tinggi_genangan')" min="0" placeholder="Contoh: 45" required />
                            <x-input-error :messages="$errors->get('tinggi_genangan')" class="mt-2" />
                        </div>

                        <div>
                            <x-input-label for="tanggal_kejadian" :value="__('Tanggal Kejadian')" />
                            <x-text-input id="tanggal_kejadian" class="block mt-1 w-full" type="date" name="tanggal_kejadian"
                                          :value="old('tanggal_kejadian', date('Y-m-d'))" required />
                            <x-input-error :messages="$errors->get('tanggal_kejadian')" class="mt-2" />
                        </div>

                        <div class="flex items-center justify-between pt-2">
                            <p class="text-sm text-gray-500">
                                Tidak perlu login untuk mengirim laporan.
                            </p>

                            <x-primary-button>
                                {{ __('Kirim Laporan') }}
                            </x-primary-button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

Halaman daftar laporan menampilkan card laporan, badge status, lokasi, nama pelapor, tinggi genangan, tanggal kejadian, dan waktu laporan masuk. Badge status ditentukan dari tinggi genangan: Waspada untuk di bawah 30 cm, Siaga untuk 30 sampai 70 cm, dan Awas untuk di atas 70 cm. Halaman ini dilindungi middleware auth. Saya juga menambahkan komponen alert untuk menampilkan pesan berhasil setelah laporan dikirim. Komponen alert memakai class Tailwind default dan hanya muncul kalau session success terisi.

```blade
<x-app-layout>
    <x-slot name="header">
        <div class="flex flex-wrap items-center justify-between gap-3">
            <h2 class="font-semibold text-xl text-gray-800 leading-tight">
                {{ __('Daftar Laporan Banjir') }}
            </h2>

            <a href="{{ route('laporan.create') }}"
               class="inline-flex items-center px-4 py-2 bg-gray-800 border border-transparent rounded-md font-semibold text-xs text-white uppercase tracking-widest hover:bg-gray-700 focus:bg-gray-700 active:bg-gray-900 focus:outline-none focus:ring-2 focus:ring-indigo-500 focus:ring-offset-2 transition ease-in-out duration-150">
                Buat Laporan
            </a>
        </div>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8 space-y-6">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    <p class="text-sm text-gray-600">
                        Halaman ini hanya dapat diakses oleh petugas BPBD yang sudah login.
                        Total laporan tersimpan: <strong>{{ $laporans->count() }}</strong>.
                    </p>
                </div>
            </div>

            @forelse ($laporans as $laporan)
                @include('partials.laporan-card', ['laporan' => $laporan])
            @empty
                <x-alert type="warning" message="Belum ada laporan banjir yang masuk." />
            @endforelse
        </div>
    </div>
</x-app-layout>
```

3.7 Navigasi untuk Guest dan Petugas

File layouts/navigation.blade.php adalah file yang paling terasa dampaknya. Navigasi Breeze bawaan menganggap semua pengunjung sudah login, karena memanggil Auth::user()->name tanpa pengecekan. Akibatnya, halaman publik LaporBanjir yang memakai x-app-layout bisa error saat dibuka tanpa login. Saya membungkus bagian kanan navigasi dengan @auth dan @else. Kalau sudah login, muncul tautan Dashboard, Daftar Laporan, nama pengguna, Profil Petugas, dan Log Out. Kalau belum login, muncul tautan Buat Laporan, Login, dan Register.

```blade
            @auth
                <!-- Settings Dropdown -->
                <div class="hidden sm:flex sm:items-center sm:ms-6">
                    <x-dropdown align="right" width="48">
                        <x-slot name="trigger">
                            <button class="inline-flex items-center px-3 py-2 border
border-transparent text-sm leading-4 font-medium rounded-md text-gray-500 bg-white
hover:text-gray-700 focus:outline-none transition ease-in-out duration-150">
                                <div>{{ Auth::user()->name }}</div>

                                <div class="ms-1">
                                    <svg class="fill-current h-4 w-4"
xmlns="http://www.w3.org/2000/svg" viewBox="0 0 20 20">
                                        <path fill-rule="evenodd" d="M5.293 7.293a1 1 0
011.414 0L10 10.586l3.293-3.293a1 1 0 111.414 1.414l-4 4a1 1 0 01-1.414 0l-4-4a1 1 0 010-
1.414z" clip-rule="evenodd" />
                                    </svg>
                                </div>
                            </button>
                        </x-slot>

                        <x-slot name="content">
                            <x-dropdown-link :href="route('profile.edit')">
                                {{ __('Profil Petugas') }}
                            </x-dropdown-link>

                            <!-- Authentication -->
                            <form method="POST" action="{{ route('logout') }}">
                                @csrf

                                <x-dropdown-link :href="route('logout')"
                                        onclick="event.preventDefault();
                                                    this.closest('form').submit();">
                                    {{ __('Log Out') }}
                                </x-dropdown-link>
                            </form>
                        </x-slot>
                    </x-dropdown>
                </div>
            @else
                <!-- Menu untuk pengguna yang belum login -->
                <div class="hidden sm:flex sm:items-center sm:ms-6 space-x-4">
                    <a href="{{ route('login') }}" class="text-sm font-medium text-gray-
700 hover:text-gray-900">
                        {{ __('Login') }}
                    </a>

                    <a href="{{ route('register') }}" class="text-sm font-medium text-
gray-700 hover:text-gray-900">
                        {{ __('Register') }}
                    </a>
                </div>
            @endauth
```

Menu mobile memakai struktur yang sama. Tombol Log Out tetap memakai form POST dengan @csrf. Logo diarahkan ke halaman form laporan, bukan ke welcome, supaya warga langsung sampai ke halaman yang berguna. Perubahan ini hanya menyentuh markup Blade. File CSS Breeze tidak saya ubah.

3.8 Seeder Akun Petugas

Seeder akun ada di database/seeders/PetugasSeeder.php. Saya memakai Faker dengan locale id_ID untuk membuat lima nama dan email dummy. Password semua akun dummy adalah password. Akun admin memakai nama Admin BPBD Kabupaten Bandung, email

admin@example.com, dan password admin123. Setiap password di-hash dengan Hash::make() sebelum disimpan. Saya tidak menyimpan password asli di database.

```php
<?php

namespace Database\Seeders;

use Faker\Factory as Faker;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;

class PetugasSeeder extends Seeder
{
    /**
     * Membuat 5 akun petugas dummy dan 1 akun admin BPBD.
     */
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        // Lima akun petugas dummy dengan password "password".
        for ($i = 0; $i < 5; $i++) {
            DB::table('users')->insert([
                'name' => $faker->name,
                'email' => $faker->unique()->safeEmail,
                'password' => Hash::make('password'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }

        // Akun admin BPBD dengan email dan password tetap untuk pengujian.
        DB::table('users')->insert([
            'name' => 'Admin BPBD Kabupaten Bandung',
            'email' => 'admin@example.com',
            'password' => Hash::make('admin123'),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

DatabaseSeeder memanggil PetugasSeeder dan LaporanSeeder. Urutannya penting karena data user harus siap sebelum pengujian login. Saya menjalankan php artisan migrate:fresh --seed setiap kali ingin mengembalikan data ke kondisi awal. Cara ini menghapus data lama, jadi tidak cocok untuk database produksi.

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        $this->call([
            PetugasSeeder::class,
            LaporanSeeder::class,
        ]);
    }
}
```

3.9 Menjalankan Aplikasi dan Pengujian

Setelah semua file siap, saya menjalankan empat perintah. migrate:fresh --seed untuk membangun ulang database, serve untuk menjalankan server di port 8123, npm run build untuk mengompilasi asset, dan test untuk menjalankan test suite. Saya memakai port 8123 karena port 8000 kadang dipakai proses lain. Alamat aplikasi menjadi http://127.0.0.1:8123.

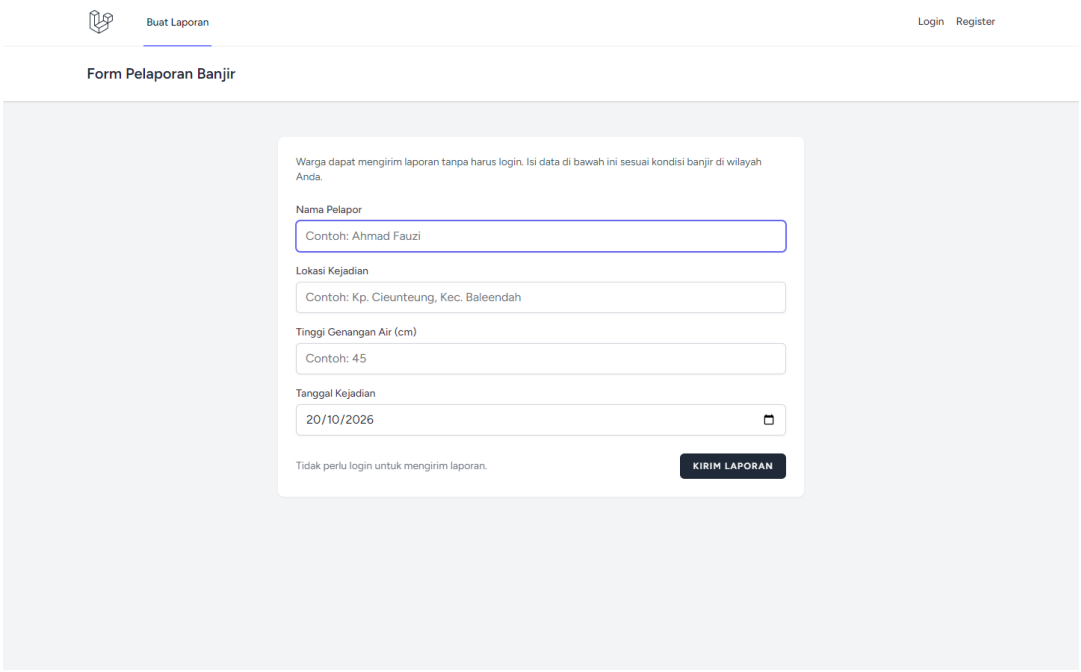


Hasil php artisan test adalah 30 passed dengan 81 assertions. Test mencakup alur autentikasi Breeze dan halaman utama LaporBanjir. Saya juga memeriksa database lewat phpMyAdmin. Tabel users berisi enam baris: satu admin dan lima petugas dummy. Tabel laporan awalnya berisi 20 baris dari LaporanSeeder. Setelah saya mencoba form publik sekali, jumlahnya bertambah menjadi 21 baris.

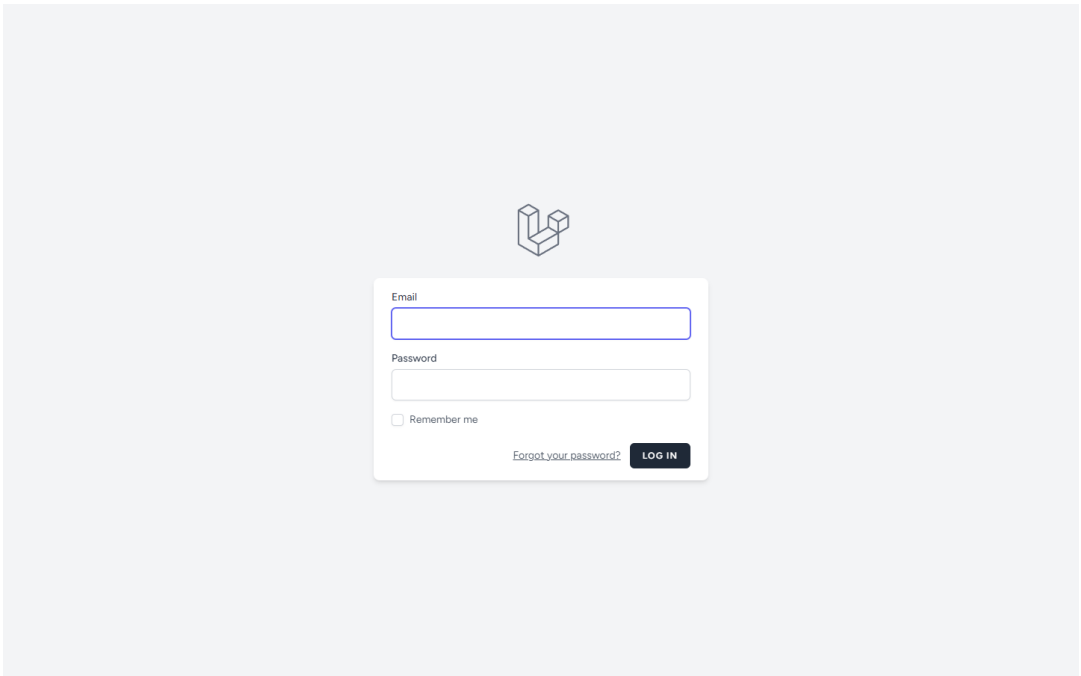
Server sempat berhenti setelah composer install. Saya menjalankannya ulang dengan php artisan serve. Setelah view diubah, asset Tailwind juga perlu dibangun ulang dengan npm run build. Dua hal ini saya catat sebagai kendala operasional, bukan bug aplikasi.

4. Hasil dan Dokumentasi Program

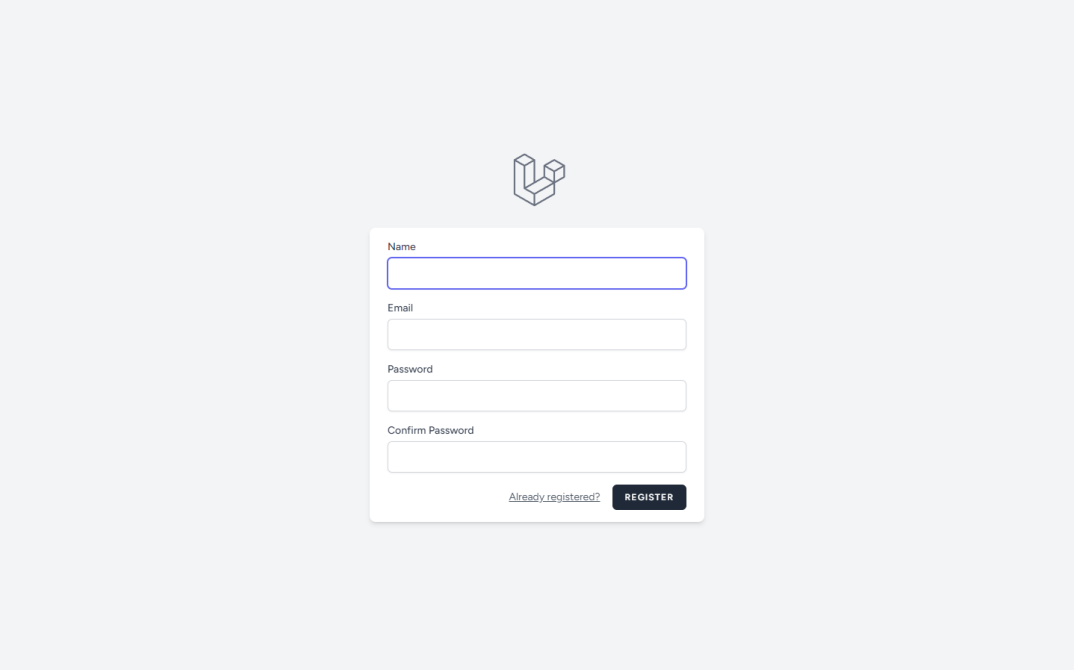
Bagian ini menampilkan hasil akhir aplikasi. Semua gambar diambil dari aplikasi yang berjalan di http://127.0.0.1:8123, bukan mockup. Urutannya mengikuti alur yang saya uji: form publik, redirect, register, login, dashboard, daftar laporan, profil, konfirmasi, dan pemeriksaan database.



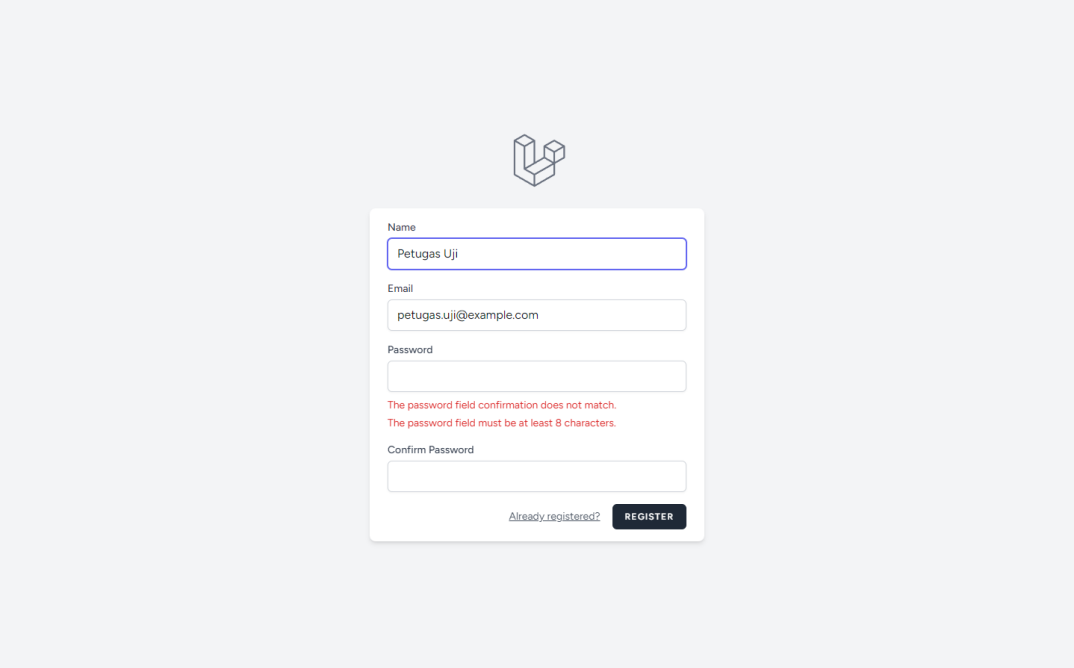
Gambar 1 menampilkan form LaporBanjir yang bisa dibuka tanpa login. Di navigasi terlihat tautan Buat Laporan, Login, dan Register. Pada form ada keterangan bahwa warga tidak perlu membuat akun. Tombol Kirim Laporan mengirim data ke route laporan.store.



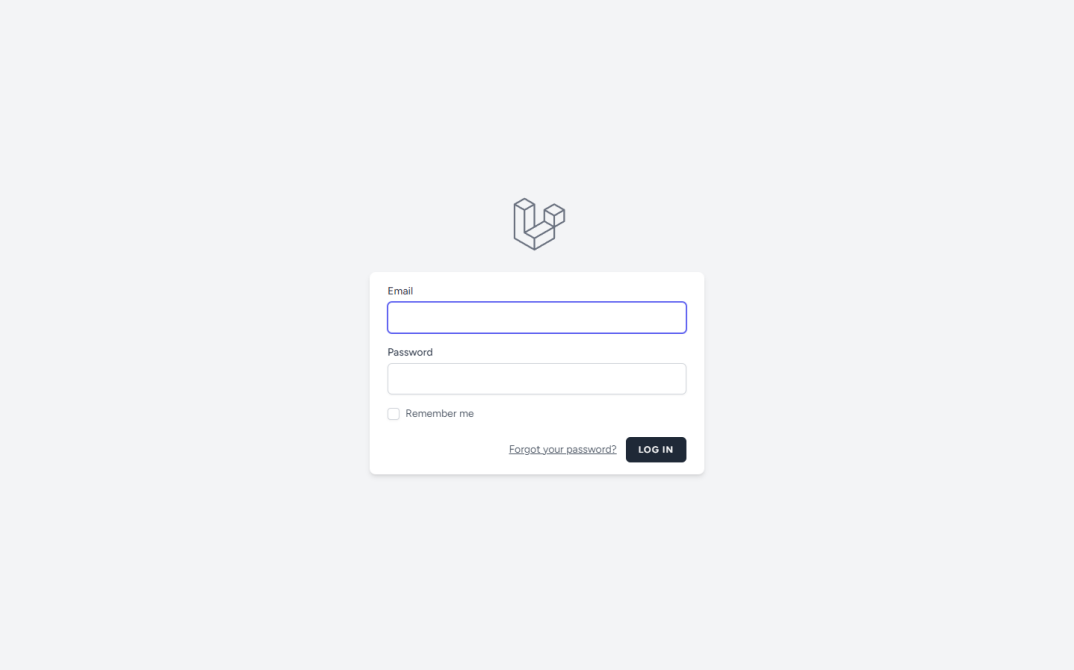
Gambar 2 menampilkan halaman login setelah saya membuka http://127.0.0.1:8123/laporan tanpa sesi. Alamat browser berubah menjadi /login. Ini menunjukkan middleware auth bekerja. Setelah login, saya diarahkan kembali ke halaman yang tadi diminta.



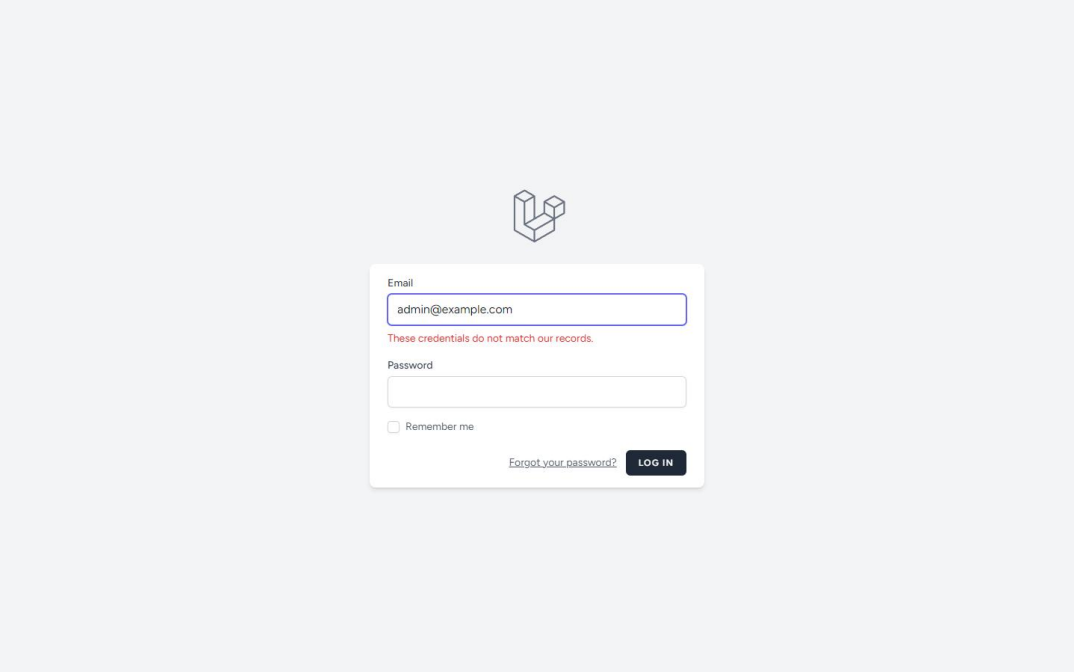
Gambar 3 menampilkan halaman register Breeze. Field Name, Email, Password, dan Confirm Password masih memakai tampilan default. Di bawah form ada tautan Already registered? ke halaman login.



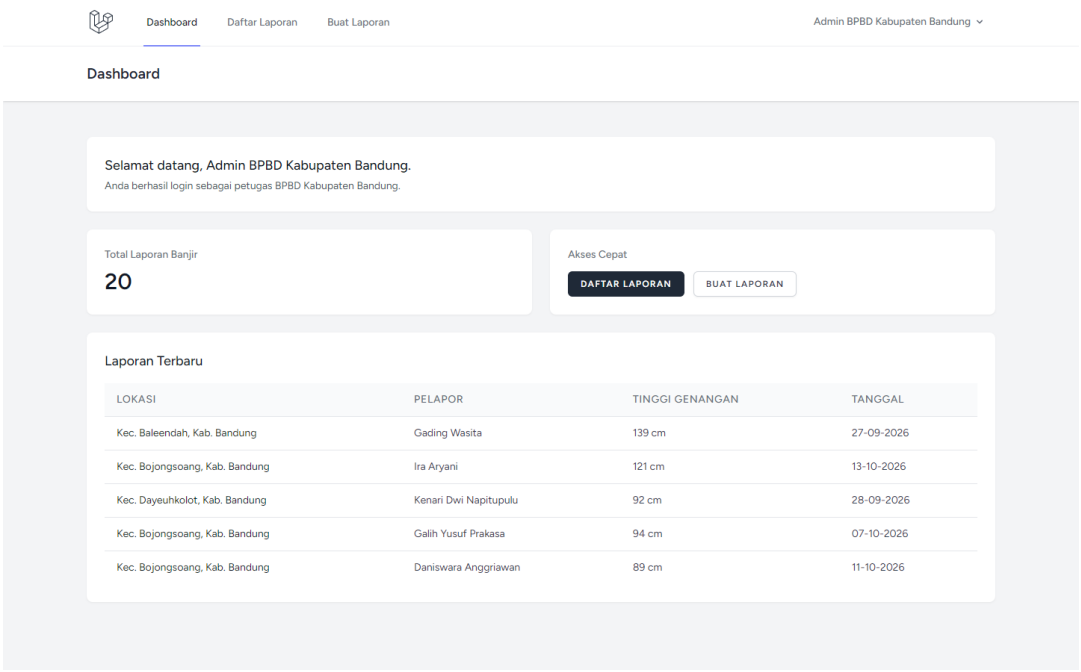
Gambar 4 menampilkan validasi saat password diisi 123 dan konfirmasinya diisi 456. Muncul pesan konfirmasi password tidak cocok dan panjang password minimal delapan karakter. Data tidak tersimpan dan pengguna tetap di halaman register.



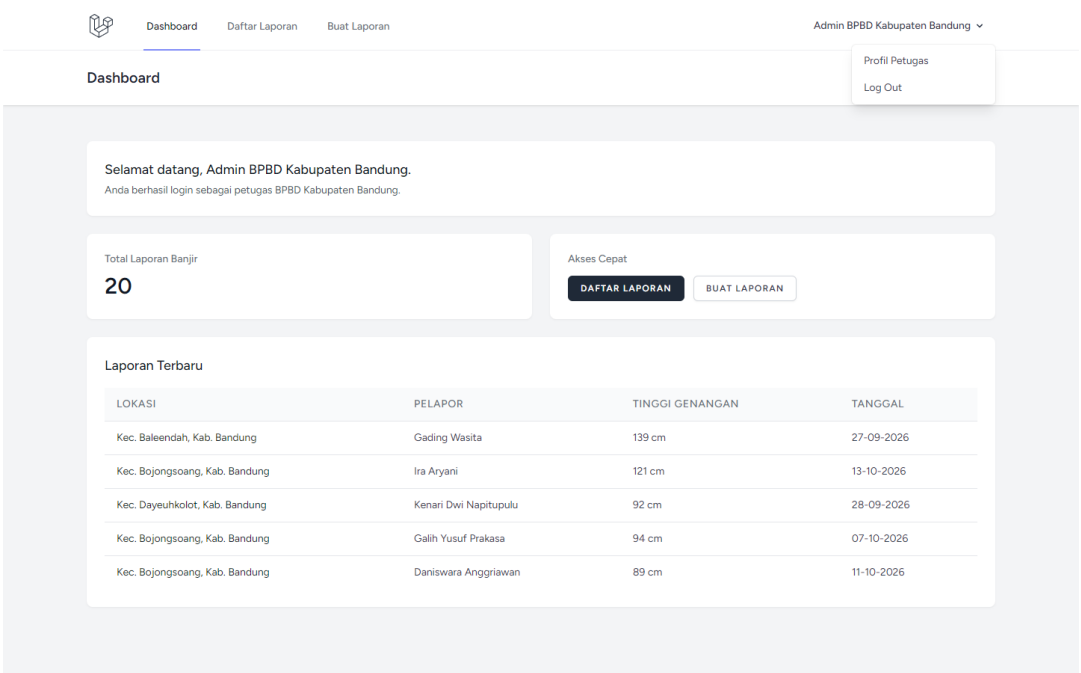
Gambar 5 menampilkan halaman login dengan Email, Password, Remember me, dan tautan Forgot your password. Di bawahnya ada tautan Register.



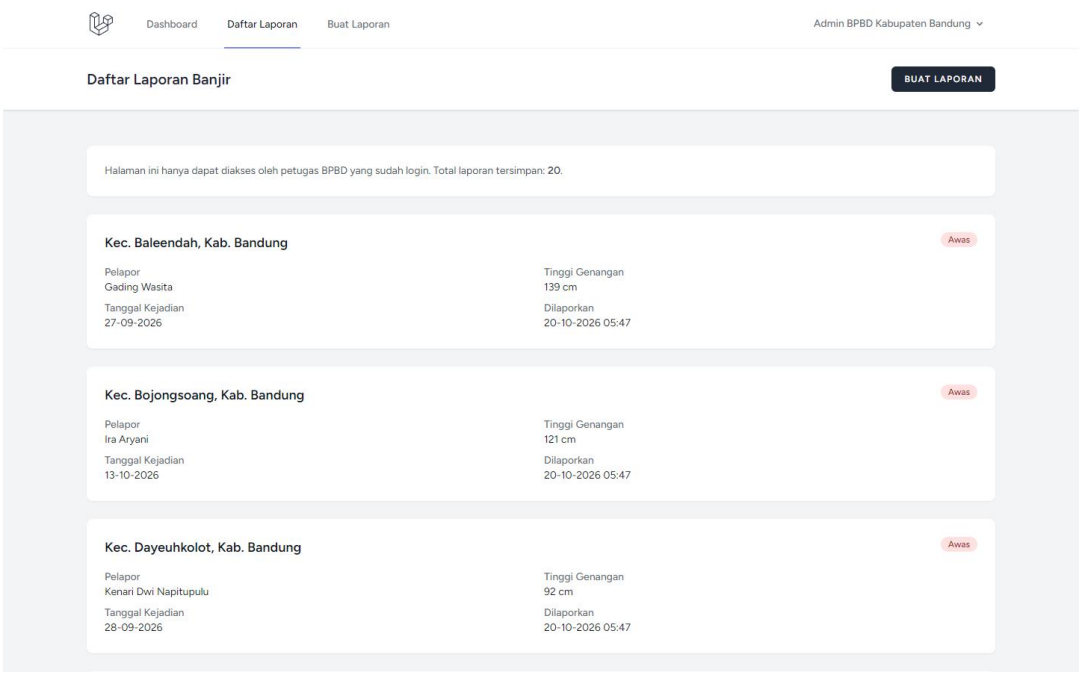
Gambar 6 menampilkan pesan These credentials do not match our records. Pesan itu muncul setelah saya memasukkan password yang salah untuk admin@example.com. Halaman tetap di /login dan tidak ada sesi yang dibuat.



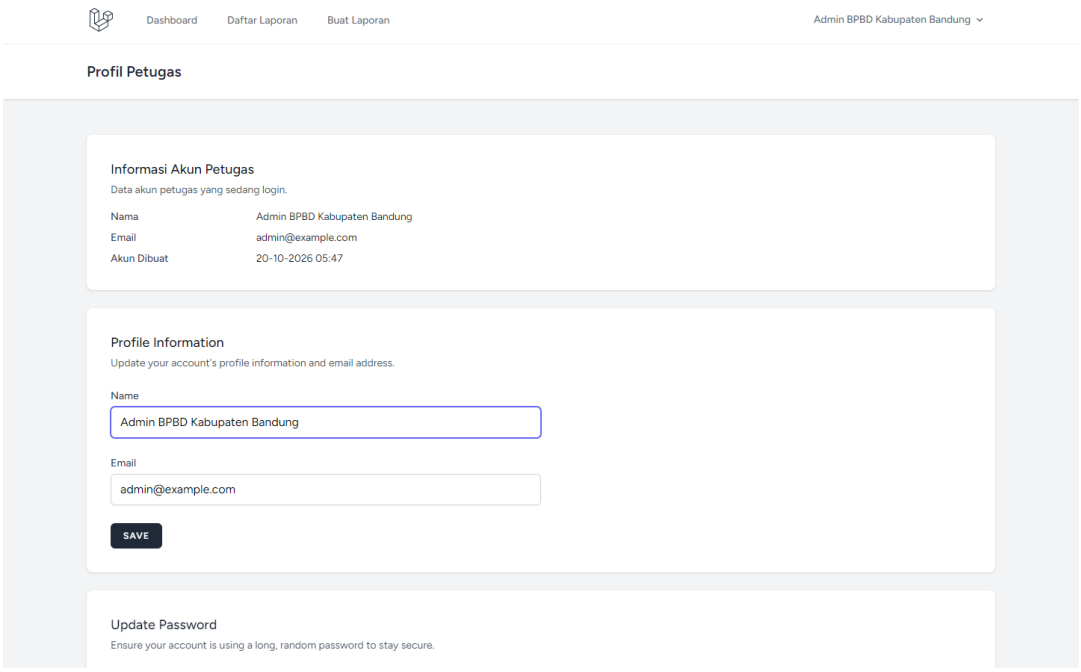
Gambar 7 menampilkan dashboard admin setelah login. Sapaan menyebut nama Admin BPBD Kabupaten Bandung. Kartu Total Laporan Banjir menunjukkan angka 20, di sebelahnya ada tombol Daftar Laporan dan Buat Laporan. Tabel Laporan Terbaru menampilkan lima laporan dengan kolom Lokasi, Pelapor, Tinggi Genangan, dan Tanggal. Pada saat gambar diambil, jumlah laporan masih 20 karena data uji form publik belum ditambahkan.



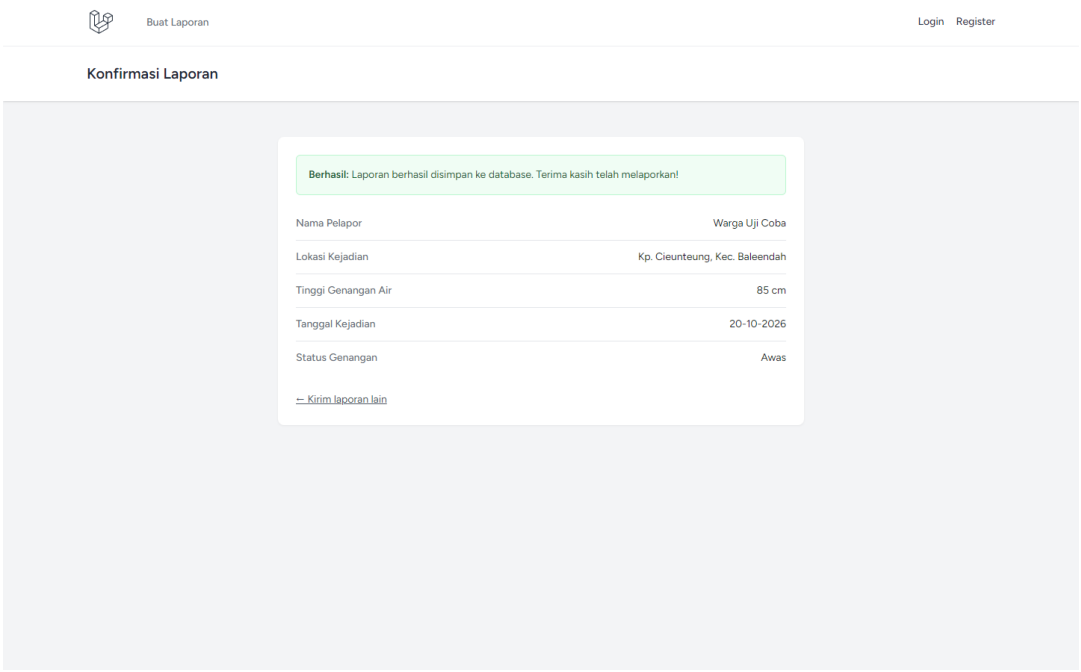
Gambar 8 menampilkan dropdown di pojok kanan atas. Isinya Profil Petugas dan Log Out. Menu ini hanya muncul untuk pengguna yang sudah login.



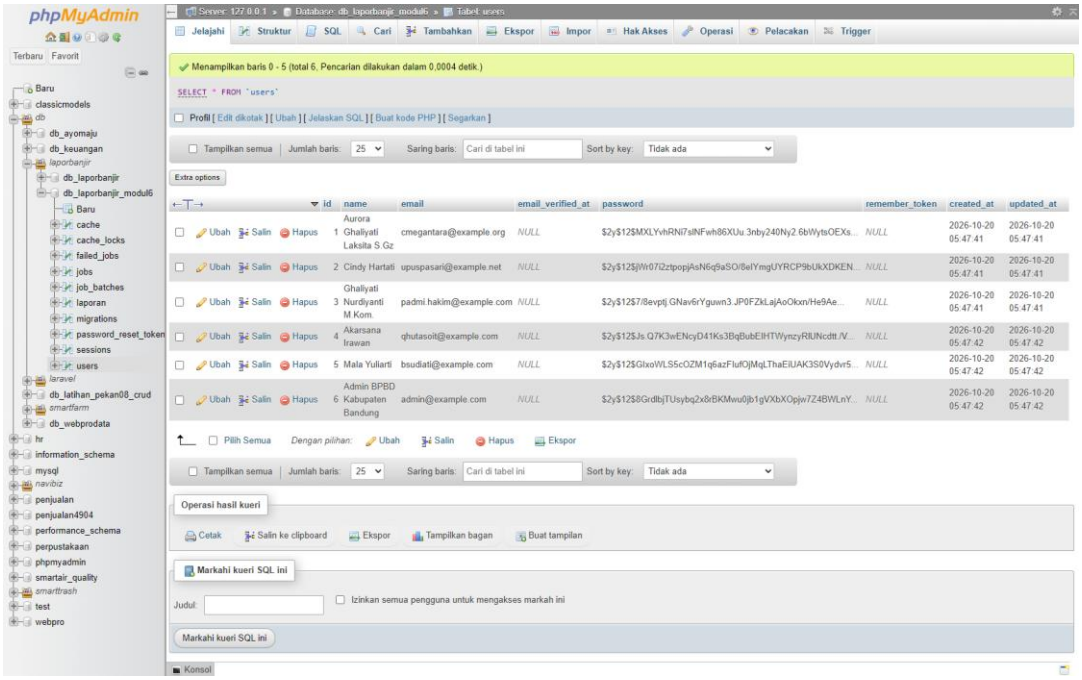
Gambar 9 menampilkan halaman Daftar Laporan yang dilindungi. Setiap card berisi lokasi kejadian, badge status Awas, nama pelapor, tinggi genangan, tanggal kejadian, dan waktu laporan masuk. Halaman ini tidak bisa dibuka tanpa login.



Gambar 10 menampilkan halaman profil petugas. Card Informasi Akun Petugas berisi nama, email, dan tanggal akun dibuat. Di bawahnya ada form Update Password dan Delete Account bawaan Breeze.



Gambar 11 menampilkan halaman konfirmasi setelah laporan dikirim. Data laporan yang baru masuk ditampilkan kembali. Karena saya membuka halaman ini sebagai guest, tautan Lihat Daftar Laporan tidak muncul. Tautan itu hanya muncul untuk petugas yang sudah login.



Gambar 12 menampilkan tabel users di phpMyAdmin. Ada enam baris: satu admin dan lima petugas. Kolom password berisi hash bcrypt yang diawali $2y$12$, bukan password asli. Ini membuktikan proses hashing berjalan.

5. Analisis dan Pembahasan

Perubahan paling terasa setelah Breeze dipasang adalah pembagian hak akses. Sebelumnya semua halaman LaporBanjir terbuka. Sekarang form pelaporan dan halaman konfirmasi tetap publik, sedangkan dashboard, daftar laporan, dan profil hanya bisa dibuka setelah login. Pembagian ini sesuai kebutuhan aplikasi. Warga tidak perlu akun untuk melapor, tetapi data laporan tidak bisa dilihat sembarang orang.

Breeze mempercepat pekerjaan karena route, controller, request, view, dan test sudah dibuat. Saya tidak perlu menulis ulang proses register, login, logout, reset password, dan update profil. Meskipun begitu, saya tetap membaca kode yang dihasilkan. Tanpa memahami Auth::attempt(), Hash::make(), session regenerate, dan middleware auth, saya akan sulit menyesuaikan halaman LaporBanjir. Bagian yang paling sering saya buka adalah LoginRequest dan AuthenticatedSessionController.

Password disimpan sebagai hash bcrypt. Saat login, Auth::attempt() mencocokkan password yang dikirim dengan hash di database. Password asli tidak pernah disimpan. Sesi login disimpan di server dan diikat ke cookie browser. Setelah login berhasil, session ID di-regenerate. Saat logout, session dihapus dan token CSRF dibuat ulang. Rangkaian ini membuat sesi lama tidak bisa dipakai lagi.

Pemisahan halaman publik dan privat juga memengaruhi navigasi. Navigasi Breeze bawaan memakai Auth::user() tanpa pengecekan, sehingga halaman publik yang memakai layout yang sama bisa error. Saya memperbaikinya dengan @auth dan @else. Kalau pengguna belum login, navigasi menampilkan Login dan Register. Kalau sudah login, navigasi menampilkan Dashboard, Daftar Laporan, dan menu profil. Perbaikan ini kecil, tetapi penting karena form publik adalah halaman pertama yang dilihat warga.

Modul ini juga menjelaskan cara membuat autentikasi manual. Pada TA6 saya tidak memakai cara itu karena instruksi tugas meminta Breeze. Kalau dibandingkan, autentikasi manual memberi kontrol penuh atas route dan tampilan, tetapi membutuhkan lebih banyak file dan pengujian. Breeze cocok dipakai di sini karena fitur yang dibutuhkan sudah lengkap dan tampilannya netral.

Kendala lain muncul saat mengubah view. Class Tailwind baru tidak langsung masuk ke CSS hasil build. Saya harus menjalankan npm run build lagi setiap kali menambah class yang belum pernah dipakai. Setelah build, tampilan langsung berubah. Saya juga sempat melihat test logout masih mengharapkan redirect ke laporan.create, padahal Breeze mengarahkan ke /. Test itu saya sesuaikan dengan perilaku Breeze. Server pengembangan juga perlu dijalankan ulang setelah composer install. Semua kendala ini sudah teratasi dan tidak mengubah alur aplikasi.

6. Kesimpulan

Autentikasi pada LaporBanjir sudah berjalan. Register, login, logout, dan profile memakai Laravel Breeze. Halaman daftar laporan dan dashboard terlindungi middleware auth, sedangkan form

pelaporan tetap publik. PetugasSeeder menyiapkan satu admin dan lima petugas dummy dengan password yang di-hash. Pengujian php artisan test menghasilkan 30 passed dan 81 assertions.

CSS Breeze tetap memakai konfigurasi default. Perubahan tampilan hanya dilakukan lewat class Tailwind pada file Blade, lalu asset dikompilasi dengan Node.js dan Vite. Data awal berisi 20 laporan dari seeder, dan satu laporan tambahan muncul setelah uji form publik. Laporan ini disusun berdasarkan project TA6, bukan project PRK_6.

7. Link GitHub

https://github.com/davinaileenskie1-hue/PABW-Semester-3